

Exercise series 9: Concurrency Control



ADReM
Advanced Database Research & Modelling
University of Antwerp

 Universiteit
Antwerpen

Problem 1:

Consider the following classes of schedules: *conflict-serializable*, *serializable*, *recoverable*, and *avoids-cascading-rollback*. For each of the following *schedules*, state which of the preceding classes it belongs to.

T1	T2
Write(X)	Read(Y) Read(X) Commit
Read(Y)	
Commit	

T1	T2	T3
Read(X)	Read(Y) Read(X) Commit	Write(X) Commit
Read(Y) Commit		

T1	T2	T3
Read(X)	Write(X) Commit	
Write(X)		
Commit		
		Write(X) Commit



Problem 1:

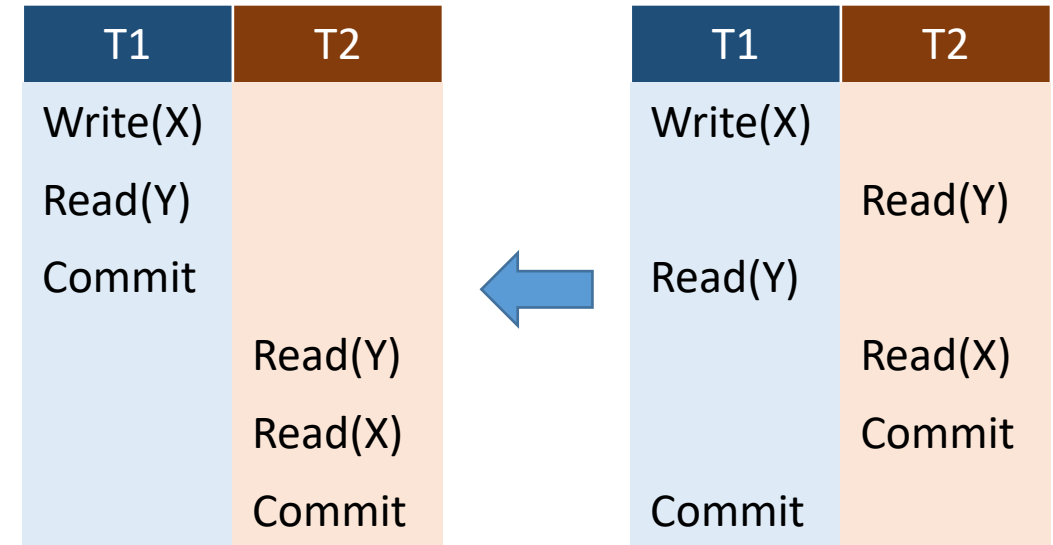
- Conflict-Serializable?
- Serializable?
- Recoverable?
- avoids-cascading-rollback?

T1	T2
Write(X)	Read(Y) Read(X) Commit
Read(Y)	
Commit	



Problem 1:

- Conflict-Serializable: Yes!
- Serializable?
- Recoverable?
- avoids-cascading-rollback?



Problem 1:

- Conflict-Serializable: Yes!
- Serializable: Yes!
- Recoverable?
- avoids-cascading-rollback?

Conflict-Serializable → Serializable

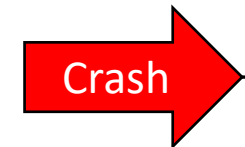
T1	T2
Write(X)	Read(Y)
Read(Y)	Read(X)
	Commit
Commit	



Problem 1:

- Conflict-Serializable: Yes!
- Serializable: Yes!
- Recoverable: No!
- avoids-cascading-rollback?

T2 has already committed
but the Read(X) in T2 does
not exist anymore

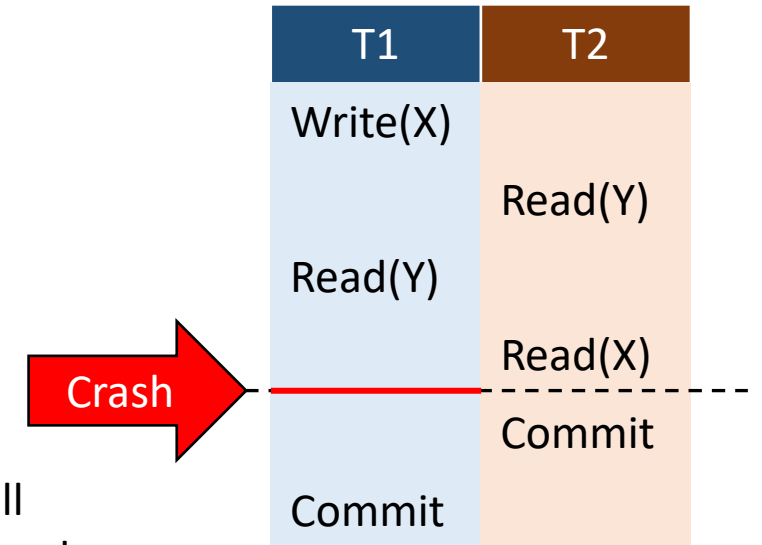


T1	T2
Write(X)	Read(Y)
Read(Y)	Read(X)
Commit	Commit



Problem 1:

- Conflict-Serializable: Yes!
- Serializable: Yes!
- Recoverable: No!
- avoids-cascading-rollback: No!



T2 must rollback as well
because it has a dirty read
on X

Problem 1:

- Conflict-Serializable?
- Serializable?
- Recoverable?
- avoids-cascading-rollback?

T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 1:

- Conflict-Serializable?
- Serializable?
- Recoverable?
- avoids-cascading-rollback?

$T1 < T3 < T2$

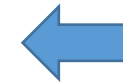
T1	T2	T3
Read(X)		
	Read(Y)	
		Write(X)
	Read(X)	
	Commit	
		Commit
Read(Y)		
Commit		



Problem 1:

- Conflict-Serializable: Yes!
- Serializable?
- Recoverable?
- avoids-cascading-rollback?

T1	T2	T3
Read(X)		
Read(Y)		
Commit		
		Write(X)
		Commit
	Read(Y)	
	Read(X)	
	Commit	

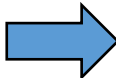


T1	T2	T3
Read(X)		
	Read(Y)	
	Read(X)	Write(X)
	Commit	
		Commit
Read(Y)		
Commit		



Problem 1:

- Conflict-Serializable: Yes!
- Serializable: Yes!
- Recoverable?
- avoids-cascading-rollback?

Conflict-Serializable  Serializable

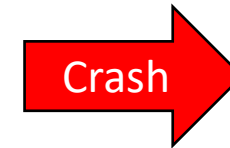
T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 1:

- Conflict-Serializable: Yes!
- Serializable: Yes!
- Recoverable: No!
- avoids-cascading-rollback?

T2 has already committed
but the Read(X) in T2 does
not exist anymore



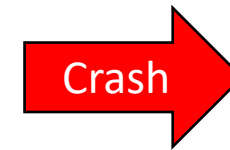
T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	
Read(Y)		Commit
Commit		



Problem 1:

- Conflict-Serializable: Yes!
- Serializable: Yes!
- Recoverable: No!
- avoids-cascading-rollback: No!

T2 must rollback as well
because it has a dirty read
on X



T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 1:

- Conflict-Serializable?
- Serializable?
- Recoverable?
- avoids-cascading-rollback?

T1	T2	T3
Read(X)	Write(X) Commit	
Write(X) Commit		
		Write(X) Commit



Problem 1:

- Conflict-Serializable?
- Serializable?
- Recoverable?
- avoids-cascading-rollback?

T1	T2	T3
Read(X)		
	Write(X)	
	Commit	
Write(X)		
Commit		
		Write(X)
		Commit



Problem 1:

- Conflict-Serializable: No!
- Serializable?
- Recoverable?
- avoids-cascading-rollback?

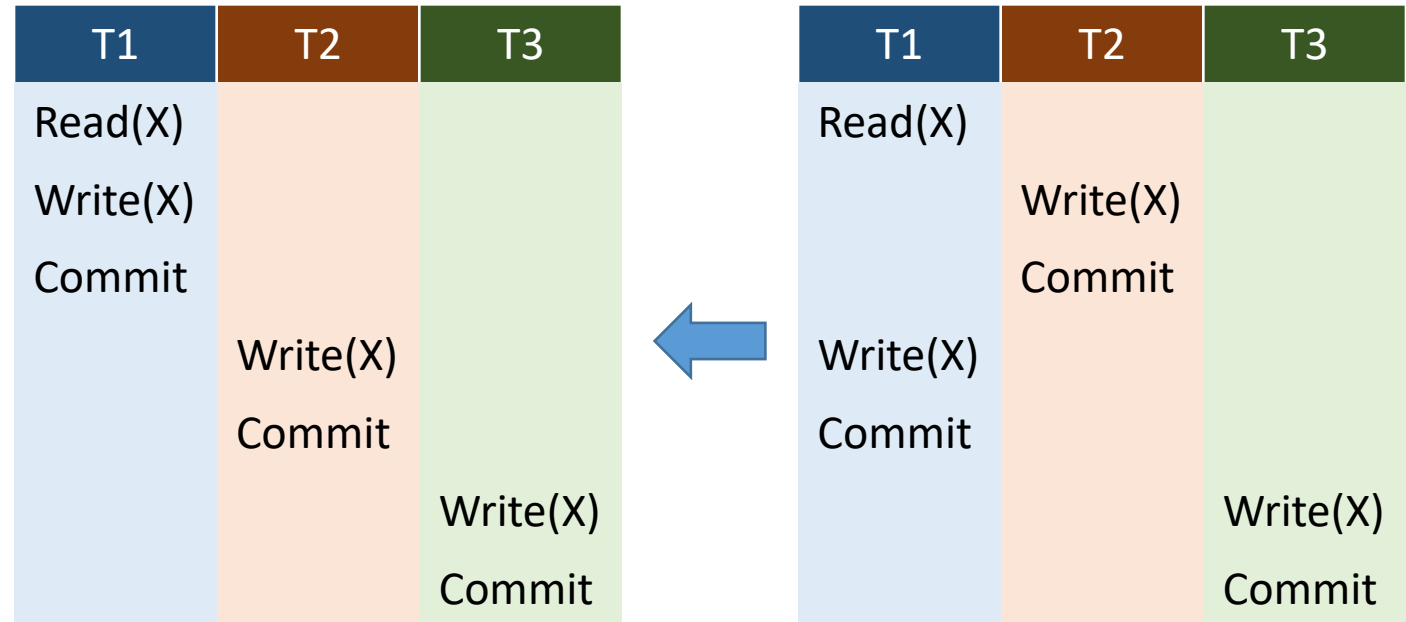
$T1 < T2 < T1$

T1	T2	T3
Read(X)		
	Write(X)	
	Commit	
Write(X)		
Commit		
		Write(X)
		Commit



Problem 1:

- Conflict-Serializable: No!
- Serializable: Yes!
- Recoverable?
- avoids-cascading-rollback?



No reads depend on Write(X) in T1 and Write(X) in T2 does not depend on a read: they can be switched without problems



Problem 1:

- Conflict-Serializable: No!
- Serializable: Yes!
- Recoverable: Yes!
- avoids-cascading-rollback?

T1	T2	T3
Read(X)	Write(X) Commit	
Write(X) Commit		
		Write(X) Commit

There are no $\text{Read}(X) > \text{Write}(X)$



Problem 1:

- Conflict-Serializable: No!
- Serializable: Yes!
- Recoverable: Yes!
- avoids-cascading-rollback: Yes!

T1	T2	T3
Read(X)	Write(X) Commit	
Write(X) Commit		
		Write(X) Commit

There are no $\text{Read}(X) > \text{Write}(X)$



Problem 2:

Consider the following concurrence control protocols:

2PL, Strict 2PL, Rigorous 2PL, Timestamp with and without Thomas Write Rule

For each of the following schedules state which of these protocols allows it, that is allows the actions to occur in exactly the order shown.

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	
	Read(X)
	Commit

T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	Commit
	Commit	
Read(Y)		
Commit		

T1	T2	T3
Read(X)	Write(X)	
Write(X)	Commit	Write(X)
Commit		
		Commit

Problem 2:

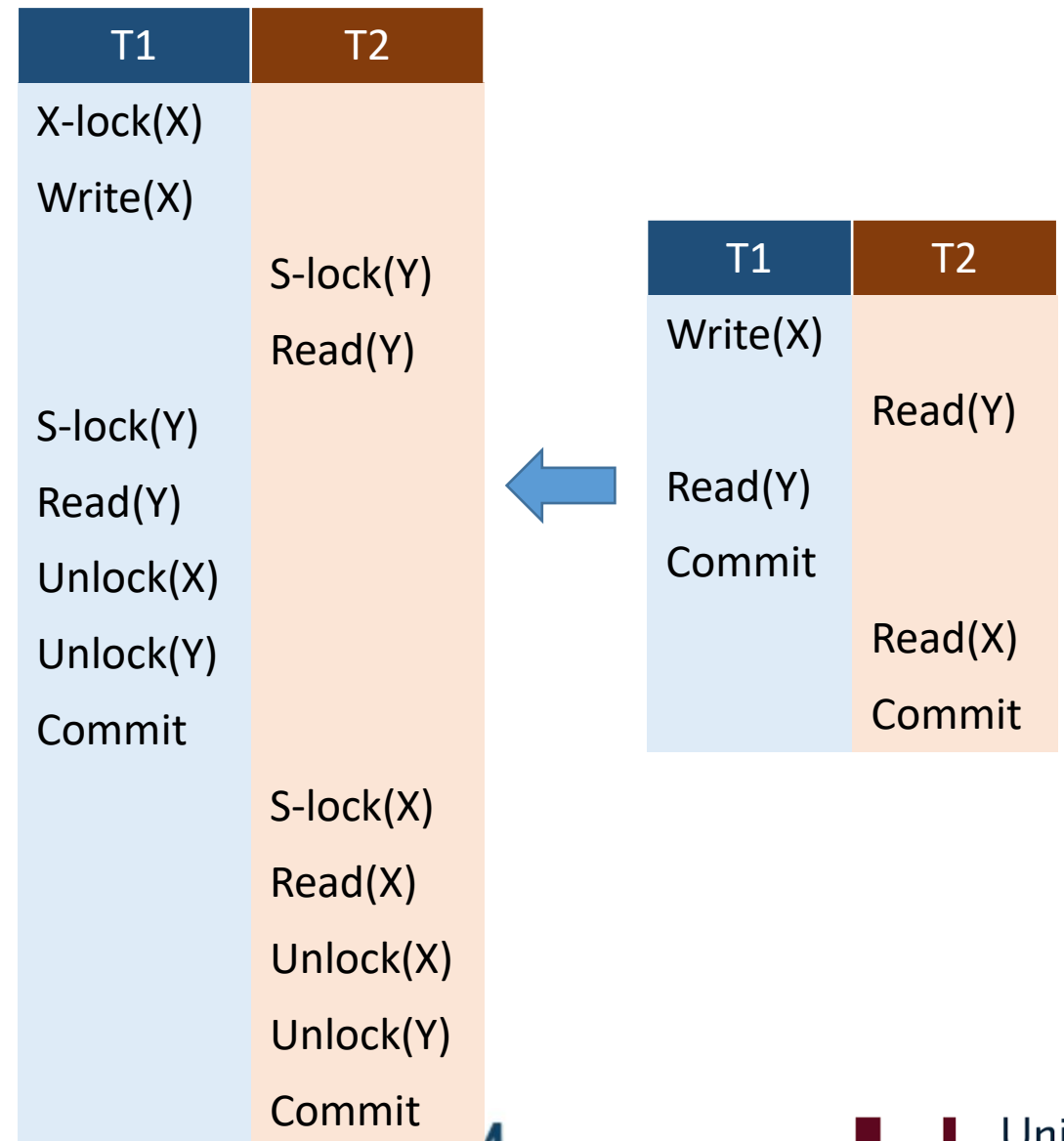
- 2PL ?
- *Strict 2PL ?*
- *Rigorous 2PL ?*
- *Timestamp ?*
- *Timestamp + Thomas?*

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit



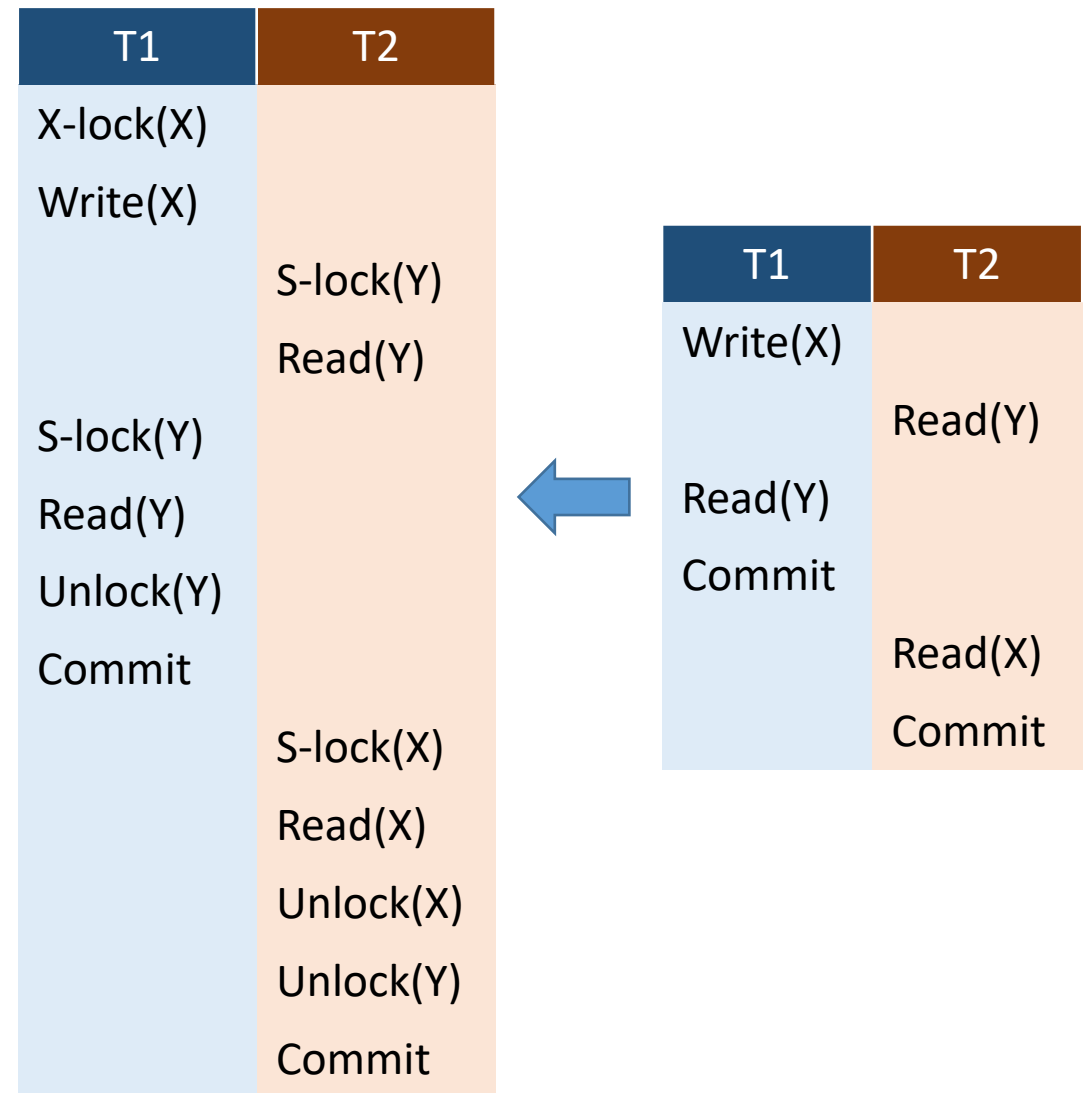
Problem 2:

- 2PL: Yes!
- *Strict 2PL ?*
- *Rigorous 2PL ?*
- *Timestamp ?*
- *Timestamp + Thomas?*



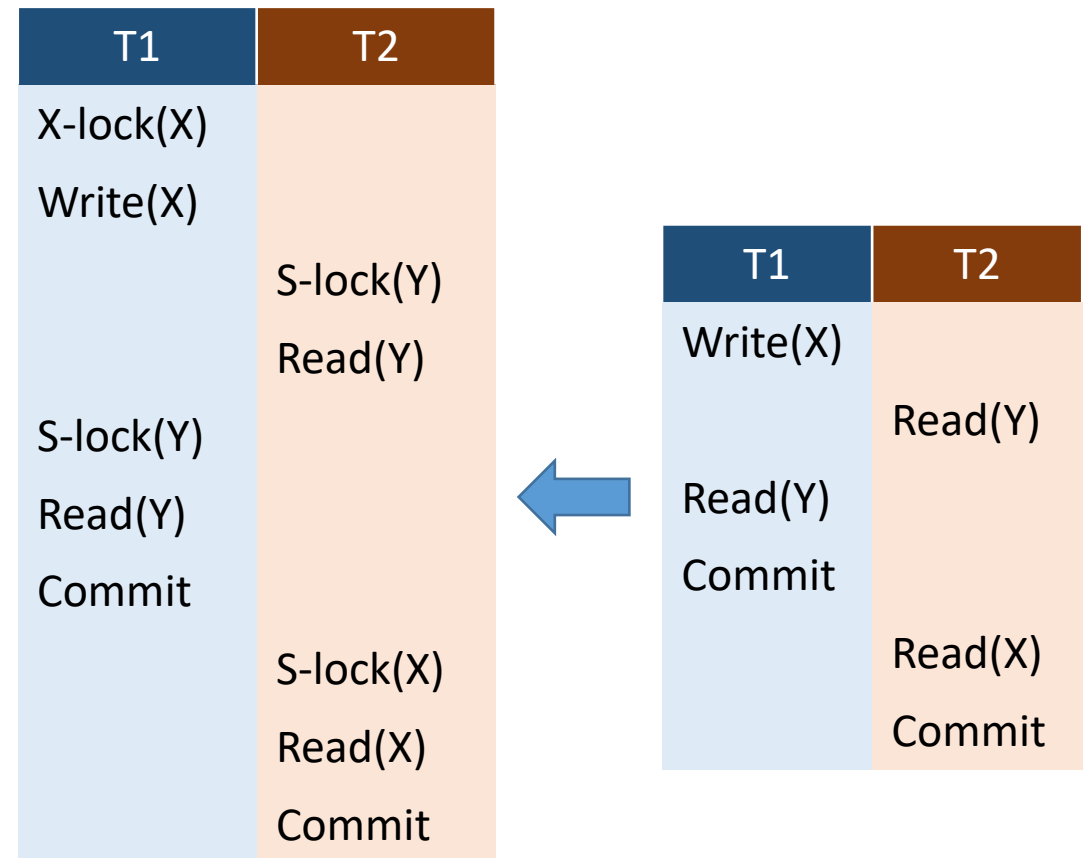
Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL* ?
- *Timestamp* ?
- *Timestamp + Thomas*?



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
0	0	0	0

→

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
0	1	0	0

→

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	
	Read(X)
	Commit



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
0	1	2	0

→

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
0	1	2	0

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit

→



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
0	1	2	0

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit

→



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp* ?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
2	1	2	0

→

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp*: Yes!
- *Timestamp + Thomas?*

X		Y	
R	W	R	W
2	1	2	0

→

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	Read(X)
	Commit



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: Yes!
- *Rigorous 2PL*: Yes!
- *Timestamp*: Yes!
- *Timestamp + Thomas*: Yes!

T1	T2
Write(X)	Read(Y)
Read(Y)	
Commit	
	Read(X)
	Commit

Timestamp → *Timestamp + Thomas*



Problem 2:

- 2PL?
- *Strict 2PL?*
- *Rigorous 2PL?*
- *Timestamp?*
- *Timestamp + Thomas?*

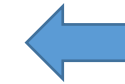
T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 2:

- 2PL: Yes!
- *Strict 2PL?*
- *Rigorous 2PL?*
- *Timestamp?*
- *Timestamp + Thomas?*

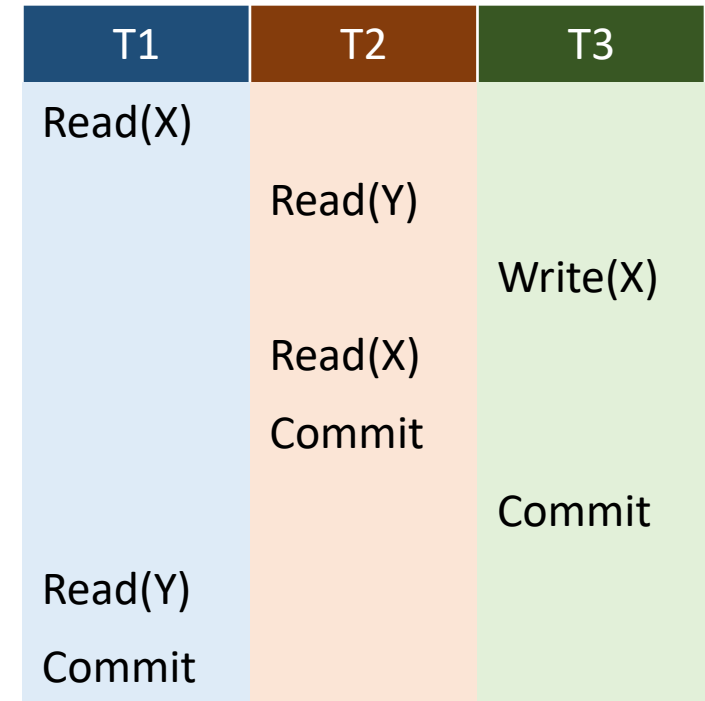
T1	T2	T3
S-lock(X) Read(X)	S-lock(Y) Read(Y)	
S-lock(Y) Unlock(X)		
		X-lock(X) Write(X) Unlock(X)
	S-lock(X) Read(X) Unlock(X) Unlock(Y) Commit	
		Commit
Read(Y) Unlock(Y) Commit		



T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X) Commit	
Read(Y) Commit		Commit

Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*?
- *Timestamp*?
- *Timestamp + Thomas*?



Conflict!



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

Rigorous 2PL $\rightarrow$ *Strict 2PL*
Therefore
 $\neg$ *Strict 2PL* $\rightarrow$ $\neg$ *Rigorous 2PL*

T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
0	0	0	0

→

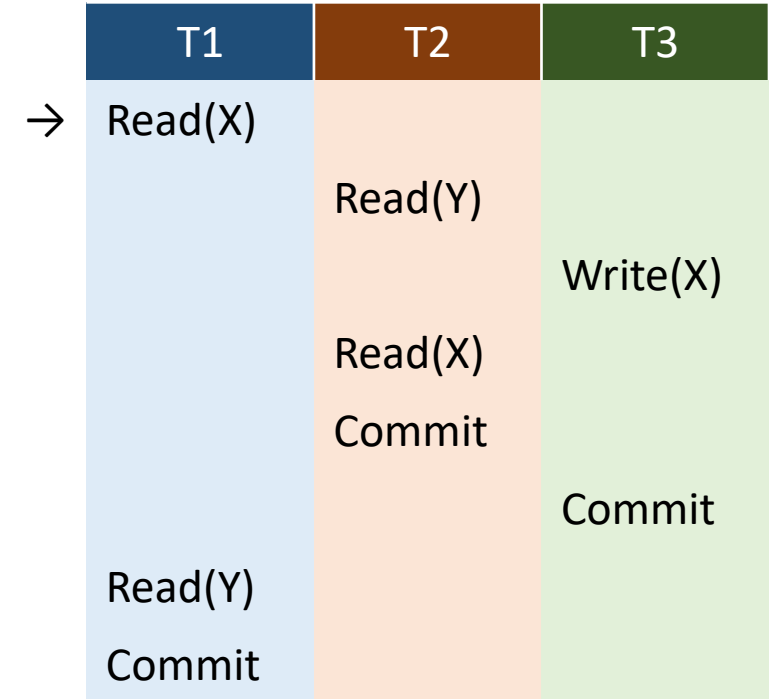
T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
1	0	0	0



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
1	0	2	0

→

T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X		Y	
R	W	R	W
1	3	2	0

→

T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*: No!
- *Timestamp + Thomas?*

X		Y	
R	W	R	W
1	3	2	0

→

T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 2:

- 2PL: Yes!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*: No!
- *Timestamp + Thomas*: No!

X		Y	
R	W	R	W
1	3	2	0

→

T1	T2	T3
Read(X)	Read(Y)	Write(X)
	Read(X)	
	Commit	Commit
Read(Y)		
Commit		



Problem 2:

- 2PL?
- *Strict 2PL?*
- *Rigorous 2PL?*
- *Timestamp?*
- *Timestamp + Thomas?*

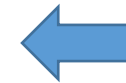
T1	T2	T3
Read(X)	Write(X) Commit	
Write(X) Commit		
		Write(X) Commit



Problem 2:

- 2PL: No!
- *Strict 2PL?*
- *Rigorous 2PL?*
- *Timestamp?*
- *Timestamp + Thomas?*

T1	T2	T3
Read(X)		
	Write(X)	
Write(X)	Commit	
Commit		
		Write(X)
		Commit



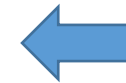
T1	T2	T3
Read(X)		
	Write(X)	
Write(X)	Commit	
Commit		
		Write(X)
		Commit



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*?
- *Timestamp*?
- *Timestamp + Thomas*?

T1	T2	T3
Read(X)		
	Write(X)	
Write(X)	Commit	
Commit		
		Write(X)
		Commit



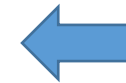
T1	T2	T3
Read(X)		
	Write(X)	
Write(X)	Commit	
Commit		
		Write(X)
		Commit



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

T1	T2	T3
Read(X)		
	Write(X)	
Write(X)	Commit	
Commit		
		Write(X)
		Commit



T1	T2	T3
Read(X)		
	Write(X)	
Write(X)	Commit	
Commit		
		Write(X)
		Commit



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X	
R	W
0	0



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X	
R	W
1	0



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X	
R	W
1	2

→

T1	T2	T3
Read(X)	Write(X) Commit	
Write(X) Commit		
		Write(X) Commit



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*?
- *Timestamp + Thomas*?

X	
R	W
1	2

→

T1	T2	T3
Read(X)	Write(X) Commit	
Write(X) Commit		
		Write(X) Commit



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*: No!
- *Timestamp + Thomas?*

X	
R	W
1	2

T1	T2	T3
Read(X)	Write(X) Commit	
Write(X) Commit		
		Write(X) Commit



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*: No!
- *Timestamp + Thomas*?

X	
R	W
1	2

→

T1	T2	T3
Read(X)	Write(X) Commit	
Commit		Write(X) Commit



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*: No!
- *Timestamp + Thomas*?

X	
R	W
1	2



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*: No!
- *Timestamp + Thomas*?

X	
R	W
1	3



Problem 2:

- 2PL: No!
- *Strict 2PL*: No!
- *Rigorous 2PL*: No!
- *Timestamp*: No!
- *Timestamp + Thomas*: Yes!

X	
R	W
1	3



Problem 3:

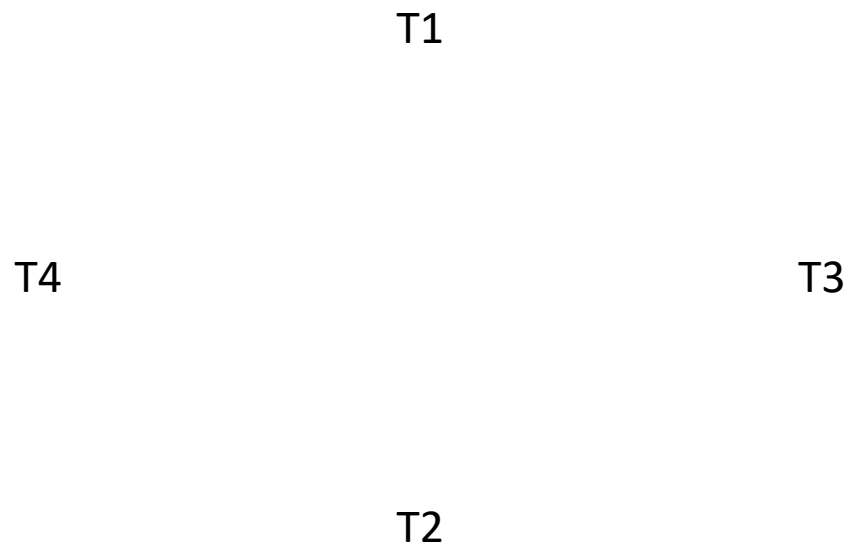
Answer the following questions:

1. Create a wait-for graph. Each transaction is a node. An arrow (or edge) is drawn between two nodes (or transaction), if transaction T_i has to wait-for transaction T_j to release a lock.
2. Does deadlock occur? How could you detect it?

T1	T2	T3	T4
S-lock(X) Read(X) S-lock(Y)	X-lock(Y) Write(Y) X-lock(W)	 S-lock(W) Read(W) X-lock(X)	 X-lock(Y)

Problem 3:

1. Create a wait-for graph.

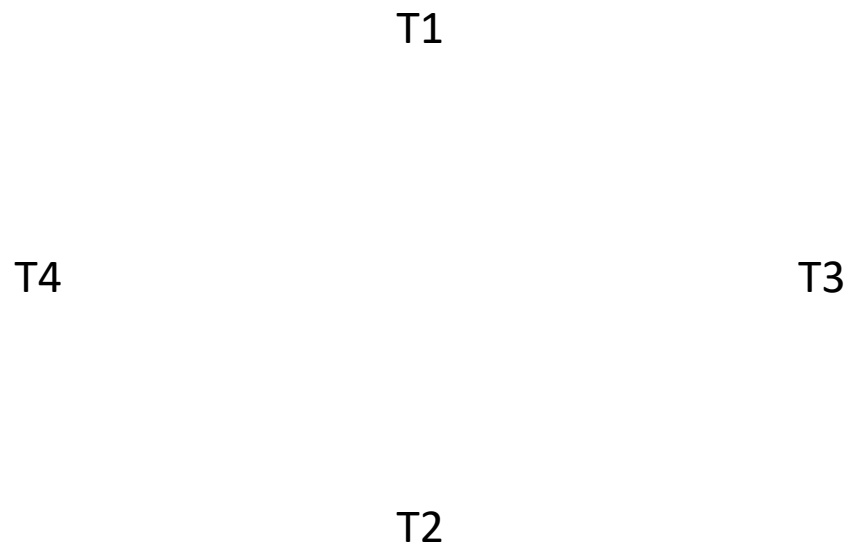


→

T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)	X-lock(X)	X-lock(Y)

Problem 3:

1. Create a wait-for graph.

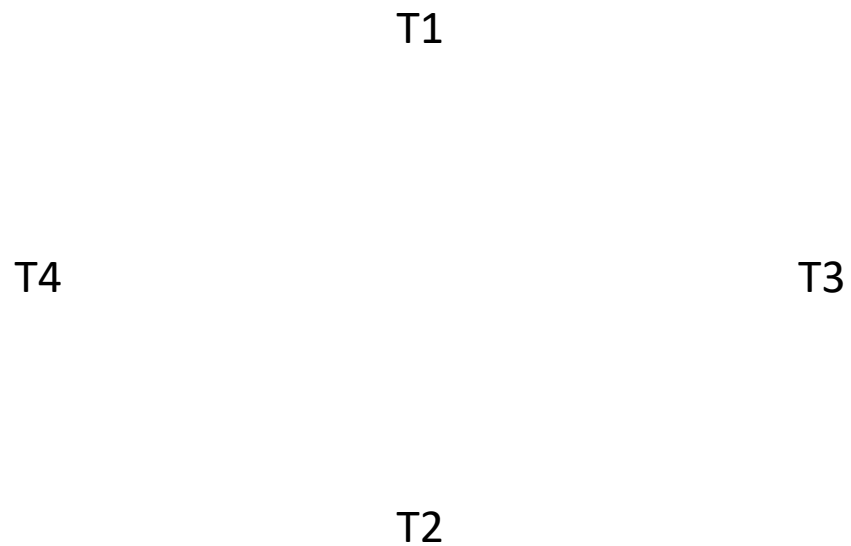


→

T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)	X-lock(X)	X-lock(Y)

Problem 3:

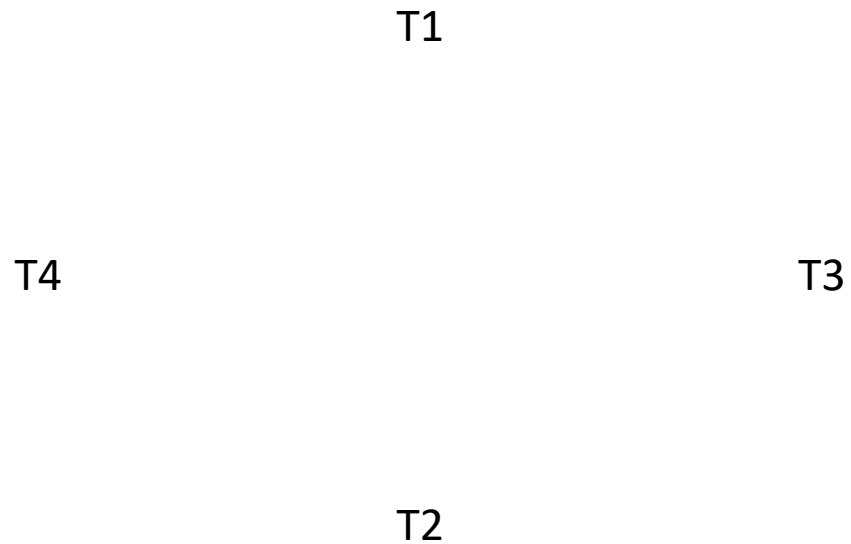
1. Create a wait-for graph.



	T1	T2	T3	T4
→	S-lock(X) Read(X) S-lock(Y)	X-lock(Y) Write(Y) X-lock(W)	 S-lock(W) Read(W) X-lock(X)	 X-lock(Y)

Problem 3:

1. Create a wait-for graph.

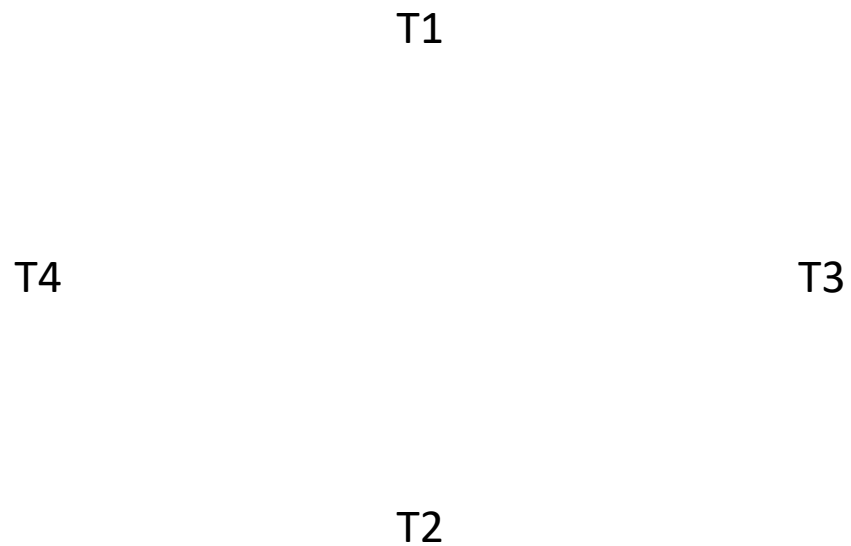


→

T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)	X-lock(X)	X-lock(Y)

Problem 3:

1. Create a wait-for graph.

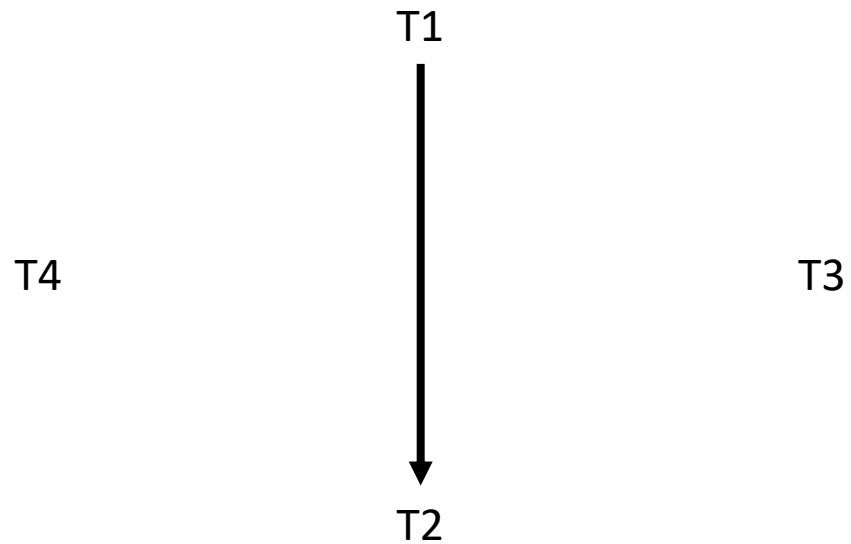


→

T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)	X-lock(X)	X-lock(Y)

Problem 3:

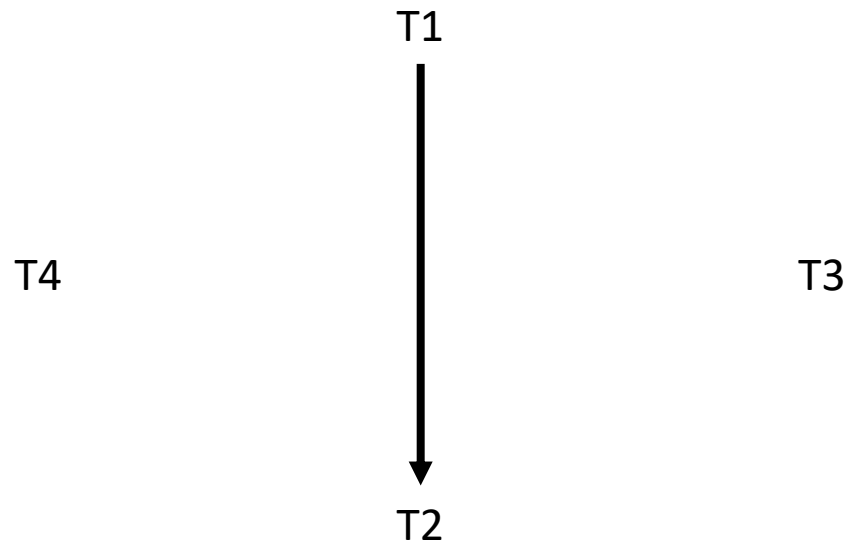
1. Create a wait-for graph.



	T1	T2	T3	T4
→	S-lock(X) Read(X) S-lock(Y)	X-lock(Y) Write(Y) X-lock(W)	 S-lock(W) Read(W) X-lock(X)	 X-lock(Y)

Problem 3:

1. Create a wait-for graph.

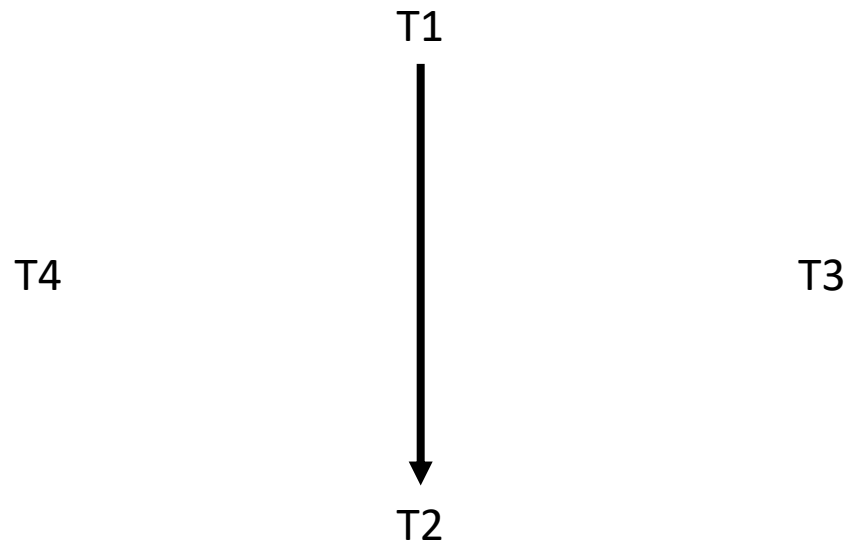


→

T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)	X-lock(X)	X-lock(Y)

Problem 3:

1. Create a wait-for graph.

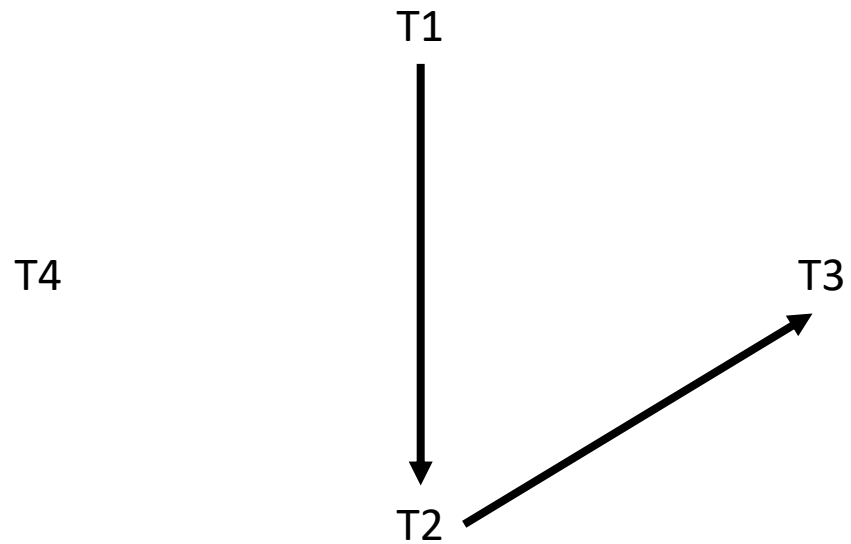


→

T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)	X-lock(X)	X-lock(Y)

Problem 3:

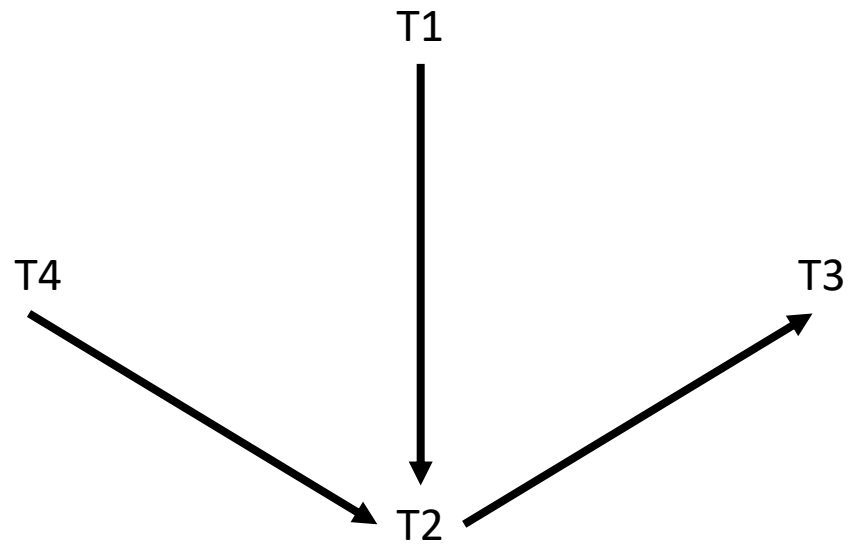
1. Create a wait-for graph.



T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)		
		X-lock(X)	X-lock(Y)

Problem 3:

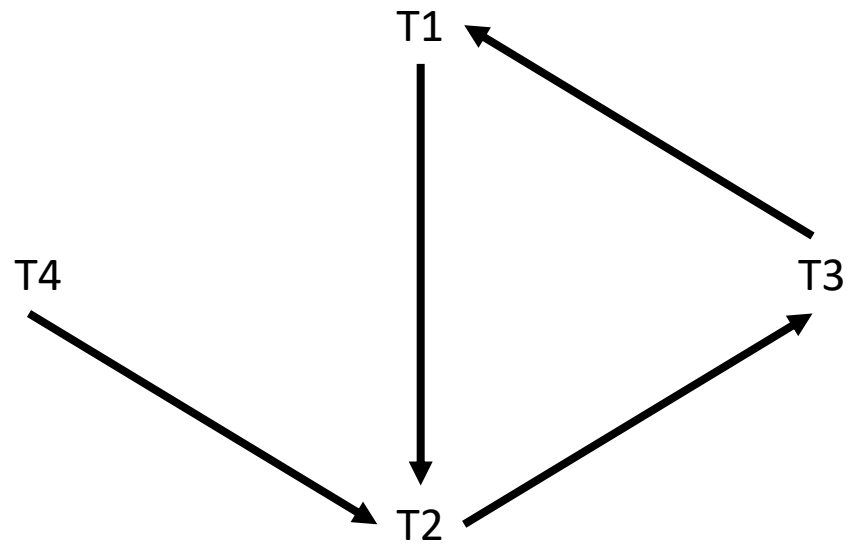
1. Create a wait-for graph.



T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)		
		X-lock(X)	
			X-lock(Y)

Problem 3:

1. Create a wait-for graph.

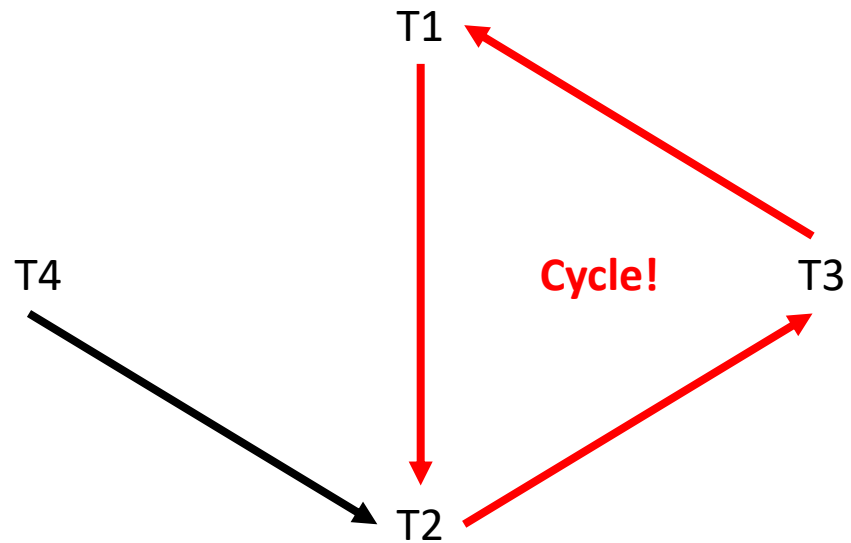


T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)		X-lock(Y)
		X-lock(X)	

→

Problem 3:

2. Does deadlock occur: Yes!
How could you detect it: Checking cycles



T1	T2	T3	T4
S-lock(X) Read(X)	X-lock(Y) Write(Y)		
S-lock(Y)		S-lock(W) Read(W)	
	X-lock(W)		X-lock(Y)
		X-lock(X)	